Neural Networks and Deep Learning (2026)
Assignment (4):

Shiraz University

Due date: 16th of Mordad 23:59 [7/Aug (2026)]

1 Graph Convolutional Network and Transformer for Molecular Representation Learning and Generation

1.1 Introduction

Molecules may be represented both as graphs and as symbolic sequences. In the graph representation, atoms are treated as nodes and chemical bonds are treated as edges. In the sequence representation, the same molecular structure can be described using SMILES or SELFIES symbols. A Graph Convolutional Network can learn molecular representations by exchanging information between bonded atoms, while a Transformer can learn sequential molecular patterns and generate new molecular sequences one token at a time. This project studies whether graph-based and sequence-based models can learn compatible representations of the same molecules and whether a small causal Transformer can generate new valid molecular structures.

1.2 Problem Statement

You will use the MoleculeNet Lipophilicity dataset, which contains 4,200 molecules with an experimental lipophilicity target. Each sample includes a SMILES string, categorical atom features, categorical bond features, and molecular bond connections. The dataset can be downloaded automatically using the PyTorch Geometric MoleculeNet loader.

The raw dataset is available at:

https://deepchemdata.s3-us-west-1.amazonaws.com/datasets/Lipophilicity.csv

The PyTorch Geometric dataset loader is documented at:

https://pytorchgeometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.MoleculeNet.html

Each molecule will be processed in two forms. A Graph Convolutional Network implemented using GCNConv will process the molecular graph, while a small Transformer will process the corresponding SELFIES sequence. The graph and sequence representations of the same molecule will be aligned using a contrastive learning objective. The Transformer will additionally be trained using next-token prediction so that it can generate new molecular sequences.

GCNConv performs normalized graph convolution with self-connections and also accepts a scalar edge weight for each connection. In this project, approximate bond orders will be supplied as edge weights so that single, double, triple, and aromatic bonds do not all contribute identically. The Transformer will generate SELFIES rather than raw SMILES. SELFIES uses a constrained molecular grammar, but a

successfully decoded SELFIES sequence must still be parsed, sanitized, and checked using RDKit before it may be reported as an accepted molecule.

The SELFIES documentation is available at:

https://selfies.readthedocs.io/en/latest/selfies.html

The RDKit documentation is available at:

https://www.rdkit.org/docs/GettingStartedInPython.html

The complete experiment should be designed to execute on a Kaggle GPU in approximately 30 minutes. Package installation and initial dataset download time may be reported separately.

1.3 Tasks

1.3.1 GCN-Based Molecular Representation

Load the Lipophilicity dataset, parse and sanitize every original SMILES string using RDKit, convert valid structures into canonical isomeric SMILES, remove duplicated canonical molecules, convert the remaining molecules into SELFIES sequences, and remove sequences containing more than 100 tokens. Create fixed training, validation, and test splits containing approximately 80%, 10%, and 10% of the usable molecules. Construct the SELFIES vocabulary using only the training split and include <pad>, <bos>, <eos>, and <unk> tokens. Report the original molecule count, removed samples, duplicated samples, usable molecule count, split sizes, vocabulary size, average atom count, average undirected bond count, and minimum, average, and maximum SELFIES sequence lengths.

Implement a three-layer molecular GCN using the GCNConv library layer or any GCN layer available. Process every categorical atom-feature field using a separate embedding table and add the resulting embeddings to create the initial atom representations. Assign approximate edge weights of 1.0, 2.0, 3.0, and 1.5 to single, double, triple, and aromatic bonds, respectively. Each GCN layer must include graph convolution, a nonlinear activation, normalization, dropout, and a residual connection. Apply global mean pooling after the final layer to obtain one graph-level molecular representation.

Pass the GCN representation through a two-layer projection network and apply L2 normalization. The resulting graph representation will be aligned with the Transformer representation of the same molecule during joint training.

1.3.2 Transformer Representation Learning and Molecular Generation

Implement a Transformer containing four Transformer layers, six self-attention heads, a hidden dimension of 192, a feed-forward dimension of 512, learned token embeddings, learned positional embeddings, and dropout. Train the Transformer using shifted SELFIES sequences. For example, if the complete sequence is <bos> [C] [C] [O] <eos>, the model must receive <bos> [C] [C] [O] as input and predict [C] [C] [O] <eos>. Apply an upper-triangular causal mask so that each position cannot access future tokens, and apply a padding mask so that <pad> tokens do not affect attention or loss calculation.

Use the final Transformer hidden state corresponding to <eos> as the molecular sequence representation. The <eos> state may attend to all preceding SELFIES tokens and therefore summarizes the complete molecular sequence. During generation, when <eos> has not yet been produced, use the final generated-token state.

Train the Transformer for 15 language-model warm-up epochs using only next-token cross-entropy. After the warm-up phase, train the GCN, Transformer, and projection layers jointly for up to 40 additional epochs. Use a batch size of approximately 256, mixed-precision training, gradient clipping, AdamW, cosine learning-rate decay, and validation-based early stopping.

Pass the GCN and Transformer representations through separate two-layer projection networks and apply L2 normalization. Treat the graph and SELFIES representation belonging to the same molecule as a positive pair and all other graph–sequence combinations in the mini-batch as negative pairs. Use a symmetric contrastive loss in both the graph-to-sequence and sequence-to-graph directions. Include an additional positive-pair cosine loss that directly encourages corresponding representations to become similar.

Train using the combined objective

$$L_{\text{total}} = L_{\text{token}} + \lambda L_{\text{contrastive}} + \gamma L_{\text{pair}},$$

using approximately $\lambda = 1.0$ and $\gamma = 0.25$ Plot the language-model warm-up loss, joint training loss, validation loss, token loss, contrastive loss, positive-pair loss, token accuracy, and perplexity.

Evaluate the learned shared representation on the test set using both graph-to-sequence and sequence-to-graph retrieval. For every test graph, rank all test SELFIES representations according to cosine similarity. Repeat the procedure in the opposite direction. Report graph-to-sequence top-1 and top-5 accuracy, sequence-to-graph top-1 and top-5 accuracy, average matching-pair cosine similarity, average nonmatching-pair cosine similarity, and the corresponding random-chance top-1 and top-5 rates.

Use the trained Transformer to generate 300 SELFIES sequences at temperatures equal to 0.8, 1.0, and 1.2. Begin every sequence with <bos>, generate one token at a time, and stop when <eos> is generated or the maximum sequence length is reached. Prevent <pad>, <bos>, and <unk> from being sampled. Prevent <eos> during the first five generation steps. Use top-k sampling with k=20 together with nucleus sampling using $p = 0.95$. The ground-truth continuation must not be provided during generation.

1.3.3 Validate every generated molecular sequence using the following procedure

a. Combine the generated symbols into a SELFIES string and decode it into SMILES. Reject the sample if decoding raises an exception or produces an empty string.

b. Parse the decoded SMILES using Chem.MolFromSmiles with sanitization disabled. Reject the sample if RDKit returns None.

c. Apply Chem.SanitizeMol. Reject the sample if sanitization raises an exception. A molecule passing parsing and sanitization is counted as RDKit-valid.

d. Verify that the molecule contains exactly one connected component using Chem.GetMolFrags.

e. Extract the set of chemical elements present in the training set. Reject generated molecules containing an element absent from the training data or any dummy atom with atomic number zero.

f. Reject molecules containing atoms with nonzero radical-electron counts.

g. Calculate the heavy-atom and molecular-weight distributions of the training set. Use the first and ninety-ninth percentiles as the accepted ranges for generated molecules. Reject molecules outside either range.

h. Calculate the maximum absolute formal charge observed in the training data. Reject generated molecules whose absolute formal charge exceeds this value.

i. Convert every accepted structure into canonical isomeric SMILES. Canonical SMILES must be used when calculating duplication and novelty because multiple strings may represent the same molecular structure.

For each temperature, report RDKit validity, acceptance rate, uniqueness, exact novelty, and structural novelty. Define RDKit validity as the fraction that passes RDKit parsing and sanitization. Define acceptance as the fraction passing every validation requirement. Define uniqueness using distinct accepted canonical structures. Define exact novelty as the fraction of unique accepted molecules absent from the training set.

Calculate a radius-two Morgan fingerprint for each accepted novel molecule and every training molecule. For each generated structure, calculate its maximum Tanimoto similarity to the training set. Count an accepted molecule as structurally novel when it is absent from the training set and its maximum training-set Tanimoto similarity is less than 0.85. Report the mean QED score of the accepted molecules as a descriptive value, but do not use QED alone to claim that a molecule is useful.

Convert the accepted novel generated molecules into PyTorch Geometric graphs. Obtain the GCN and Transformer representations of each generated molecule, calculate their graph–sequence cosine similarity, and identify the nearest training molecule in the combined representation space. Report at least ten generated examples with their graph–sequence similarity, nearest training molecule, nearest embedding similarity, and maximum fingerprint similarity. Display at least 20 accepted novel molecules using RDKit.

1.4 Report Requirements

The final report must describe dataset preparation, canonicalization, SELFIES tokenization, the GCN architecture, bond-order edge weighting, the Transformer architecture, shifted input and target sequences, positional embeddings, causal and padding masks, the two-stage training procedure, the contrastive objective, and the complete generated-molecule validation pipeline. Include dataset statistics, the SELFIES sequence-length distribution, language-model and joint-training curves, token accuracy, perplexity, retrieval results in both directions, generation metrics for all three temperatures, rejection counts for every validation stage, fingerprint-based structural-novelty results, examples of accepted and rejected outputs, nearest-neighbor results, and a grid of accepted novel molecules.(a visual grid displaying at least 20 accepted novel generated molecules using RDKit.)

The discussion should explain why global pooling is required for molecular graphs, why a scalar bond weight is only an approximation of complete bond information, why causal masking is necessary, how next-token prediction enables autoregressive generation, why a separate language-model warm-up phase is useful, how batch size affects the number of contrastive negatives, how temperature affects validity and diversity, why exact novelty does not prevent near-memorization, whether the GCN and Transformer learned compatible representations, and why SELFIES and RDKit validity do not prove molecular stability, synthesizability, safety, biological activity, environmental acceptability, or medical usefulness if you got this results.

Notes:

- Allowed programming languages: Python, MATLAB
- Any sign of cheating would result in a zero grade for this assignment.
- You should upload your submissions at:
  https://quera.org/course/add_to_course/course/27529/
  Password: nn2026
  All of the files should be in a ZIP file named in this format:
  "FirstNameFamilyName-SudentNumber.zip"
  Ex: "AmirZamani-4053040.zip"
- Your reports should be in a PDF file including: key points of your implementation, explanation of your chosen approach, reports of your final results and answers of assignment questions (if given).
- If you do not have **Anaconda** or **Python** installed, you can download Anaconda from the following link: [**DOWNLOAD LINK**]
- Use the following **domestic PyPi mirror** to install the required libraries:
  https://package-mirror.liara.ir/repository/pypi/simple
  For more information about this mirror, visit: **https://liara.ir/mirrors/pypi/**
  Example: "`pip install minisom --index-url https://package-mirror.liara.ir/repository/pypi/simple`**"**
  If this mirror doesn't work, try Runflare instead: **https://runflare.com/mirrors/pypi-mirror/**
